OLLSCOIL NA GAILLIMHE
UNIVERSITY OF GALWAY

Semester 1 Examinations 2024/2025

Course Instance Code(s)	4BCT1, 4BP1, 4BP4
Exam(s)	B. Sc. Degree (Computer Science & Information Technology) and B. Eng. (Electronic & Computer Engineering)
Module Code(s)	CT417
Module(s)	Software Engineering III
Paper No.	1
External Examiner(s)	Dr. Ramona Trestian
Internal Examiner(s)	Dr. Enda Howley *Dr. Effirul Ramlan
Instructions:	Answer any 3 questions. All questions are marked equally.
Duration	2 hours
No. of Pages	8
Discipline(s)	Computer Science
Course Co-ordinator(s)	Dr. Colm O'Riordan

Requirements:

Release in Exam Venue	Yes
MCQ Answersheet	No
Handout	None
Statistical/ Log Tables	None
Cambridge Tables	None
Graph Paper	None
Log Graph Paper	None
Other Materials	None
Graphic material in colour	No

PTO

CT417 Software Engineering III

Exam Duration: 2 Hours

Answer any THREE questions.

Question 1 [20 Marks]

a) **Scenario:**

***FoodieGo**, a global leader in the food delivery industry, operates in highly competitive markets, serving millions of users across multiple countries. Their mobile and web applications must remain **fast**, **reliable**, and **secure**, even as they release new features and updates regularly. However, FoodieGo has recently faced several challenges that have hampered their software delivery process:*

- *Despite the fast pace of their business, FoodieGo still relies on manual testing and approval processes for software updates. This has caused delays in releasing new features, particularly during periods of high user demand, such as holidays or special promotions.*

- *With multiple teams working on the same codebase, the company has frequently encountered bugs and integration conflicts when merging code from different teams. This has led to significant rework and delays.*

- *On multiple occasions, updates pushed to production have caused critical issues, leading to system downtime and negatively impacting customer experience. The lack of a robust testing environment has made it difficult to detect these issues before they reach production.*

- *Given the sensitive nature of user data (e.g., payment information, delivery addresses), FoodieGo has been under increased pressure to enhance the security of their software. However, the current manual security checks are insufficient, often delaying the detection of vulnerabilities until after release.*

As the **Solution Architect** for FoodieGo, you are tasked with designing improvements for their CI/CD pipeline to address the challenges the company is currently facing.

Based on your expertise in continuous integration and delivery, propose **FOUR** specific CI/CD pipeline improvements that would help FoodieGo resolve the issues outlined in the scenario.

[8 marks]

PTO

b) Following the CI/CD recommendation, now FoodieGo uses GitHub for version control and GitHub Actions for automating its CI/CD pipeline.

FoodieGo has several branches in its GitHub repository, and the development team follows a feature-based workflow. Currently, you are working on a new feature that is tracked in the feature-branch.

Write the Git commands to accomplish the following tasks:

- Check the status of your changes.
- Add all your changes.
- Commit your changes with the message “Implemented new feature.”
- Push your changes to the remote feature-branch.
- Merge the feature-branch into the main branch.

[5 marks]

c) FoodieGo has decided to use GitHub to deploy their codes. Below is a **GitHub Actions YAML file** that builds and tests a Java-based application used by **FoodieGo**.

```
name: Build and Test Java Application

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up JDK 11
        uses: actions/setup-java@v1
        with:
          java-version: '11'

      - name: Build with Maven
        run: mvn clean package
```

PTO

```
- name: Run tests and generate a code coverage report
        run: mvn verify

      - name: Build container image
        run: docker build -t my-app .

      - name: Push container image to registry
        run: docker push myregistry/my-app:latest
```

Explain what the build job does based on the YAML configuration provided.
[7 marks]

Question 2 [20 Marks]

FoodieGo's development team is working on improving their software testing process. As part of this, they want to integrate SAST, DAST, and unit testing into their CI/CD pipeline to ensure a more secure and reliable product.

a) Explain the difference between Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST).
[4 marks]

b) Why is it important to integrate SAST and DAST in a CI/CD pipeline?
[2 marks]

c) What is the role of code coverage in unit testing?
[2 marks]

d) FoodieGo has integrated **SonarQube** as their SAST tool. However, the development team has noticed that the **SonarQube** analysis step significantly slows down the CI/CD pipeline, causing long feedback loops.

As a solution architect, you are tasked with optimising the SAST process to reduce this delay.

- Propose TWO specific improvements to optimise the **SonarQube** analysis without reducing code quality or coverage.
- Justify why each improvement would work and explain the trade-offs.

[4 marks]

e) FoodieGo's integration of **OWASP ZAP** into their pipeline is not working as expected. After running a scan, the pipeline frequently fails, and the team struggles to find relevant vulnerability reports.

PTO

You are asked to investigate.

- Identify TWO common issues that could cause this integration to fail.
- Suggest practical solutions to resolve these issues and ensure ZAP runs successfully in the pipeline.

[4 marks]

f) The development team at FoodieGo is concerned about low test coverage in critical parts of their Java application. Although they are using JUnit for unit testing, key business logic areas are not fully covered.

- Explain how you would use a code coverage tool to identify the areas in the codebase that lack test coverage.
- Propose a strategy to increase the test coverage of these critical areas without slowing down the CI/CD pipeline.

[4 marks]

Question 3 [20 Marks]

As FoodieGo expands into the global market, its mobile app offerings need to scale to meet increasing demand. However, during this expansion, the company has discovered significant gaps in its security infrastructure. As the solution architect, you are tasked with explaining to the management team (CEO, CTO, etc) on critical security concepts to ensure they understand the potential threats and vulnerabilities faced by the company.

RFC 2828, titled "Internet Security Glossary," is a document published by the Internet Engineering Task Force (IETF). Based on the documentation,

a) Define the meaning of “vulnerability”.

[2 marks]

b) Define the meaning of “countermeasures”.

[2 marks]

Stuxnet worm, which targeted industrial control systems, and the 2017 WannaCry ransomware attack are examples of zero day vulnerabilities, which leveraged a zero-day exploit in Microsoft Windows.

c) What is zero-day vulnerability?

[2 marks]

d) Describe the 5 stages life cycle of a zero-day vulnerability.

[5 marks]

PTO

e) Some library function in C are vulnerable to buffer overflow attack. Define and explain the concept of a "buffer overflow" and how this vulnerability can be exploited by attackers.

[5 marks]

f) Discuss TWO countermeasures that can be implemented to address the issue of "buffer overflow".

[4 marks]

Question 4 [20 Marks]

FoodieGo's API currently handles user authentication and account creation but has become difficult to maintain due to redundancy and lack of scalability.

a) Explain the importance of design patterns in software development.

[4 marks]

b) Why is refactoring important for improving code reusability?

[4 marks]

c) You are tasked with refactoring key parts of the code using object-oriented design principles and design patterns to make it more maintainable and scalable.

- An abstract `User` class has already been introduced, and each user role (e.g., `Customer`, `Manager`) extends this class.
- The `authenticate()` method is `abstract` and is implemented differently for each user role.

Given the code snippet below:

```
public class UserController {

    public void authenticateCustomer(String username, String password) {
        if (username != null && password != null) {
            // Customer-specific authentication logic
            System.out.println("Authenticated Customer");
        }
    }
```

PTO

```
    public void authenticateManager(String username, String password) {
        if (username != null && password != null) {
            // Manager-specific authentication logic
            System.out.println("Authenticated Manager");
        }
    }
}
```

Refactor the authenticate methods to remove redundancy using the abstract `User` class. Your task is to restructure the `UserController` to use polymorphism, so it no longer needs separate methods for each role.

[4 marks]

d) Next, you need to refactor the account creation login method.

Given the code snippet below:

```
public class UserController {

    public void createCustomerAccount(String username, String password,
String email) {
        // Logic to create a customer account
        System.out.println("Created Customer Account");
    }

    public void createManagerAccount(String username, String password,
String storeLocation) {
        // Logic to create a manager account
        System.out.println("Created Manager Account");
    }
}
```

Refactor the `createCustomerAccount` and `createManagerAccount` methods to avoid duplication by using a Factory Pattern.

- You can assume that a `UserFactory` class has been introduced to handle the creation of different user roles.

- You are tasked with modifying the `UserController` to work with the `UserFactory`.

[4 marks]

PTO

e) Finally, you need to ensure that the `User` class can be extended for future roles.

Given the code snippet below:

```
public class UserFactory {

    public static User createUser(String role, String username, String
password, String extraInfo) {
        switch (role.toLowerCase()) {
            case "customer":
                return new Customer(username, password, extraInfo);
            case "manager":
                return new Manager(username, password, extraInfo);
            default:
                throw new IllegalArgumentException("Invalid role");
        }
    }
}
```

Refactor the `UserFactory` so that adding new roles (e.g., `Admin`) does not require modifying the `createUser` method.

- You can assume that the `Admin` class (extend the abstract `User` class) is available.
- Propose how the factory can be refactored to support new roles while adhering to the Open/Closed Principle.

[4 marks]

END